

Roger Frazier, Alex Caramanico, Dereck Roman Rosario, John Kallis

Due Date: 5/5/23

Group 1 MATH 341

Homework 8

“=====”

Problem 1

a)

We are considering the following initial value problem:

$y' = f(x, y)$ with $f(x, y) = \lambda y + (1 - \lambda)\cos x - (1 + \lambda)\sin x$, $\lambda \in \text{all real numbers}$, $y(0) = 1$.

The work below shows the first four steps of the explicit Euler method using $\lambda = -50$, $h = 0.5$, with known true solution $Y(x) = \sin(x) + \cos(x)$

1. $f(x, y) = \lambda y + (1 - \lambda)\cos x - (1 + \lambda)\sin x$, $\lambda \in \mathbb{R}$, $y(0) = 1$

a) $y_{n+1} = y_n + h f(x_n, y_n)$, $\lambda = -50$, $h = 0.5$, $y_0 = 1 \rightarrow f(x_0, y_0) = 60$

i. $x_1 = x_0 + h = 0.5$ $-50y + 51\cos x + 49\sin x = f(x, y)$

$f(x_0, y_0) = -50 + 51 = 1$

$y_1 = y_0 + h f(x_0, y_0) = 1 + 0.5 = 1.5 \rightarrow (x_1, y_1) = (0.5, 1.5)$

$\therefore x_2 = 1$

$f(x_1, y_1) = -50(1.5) + 51\cos(0.5) + 49\sin(0.5) = -6.751$

$y_2 = y_1 + h f(x_1, y_1) = 1.5 + \frac{1}{2}(-6.751) = -1.875718976 \rightarrow (x_2, y_2) = (1, -1.875718976)$

$\therefore x_3 = 1.5$

$f(x_2, y_2) = -50(-1.875718976) + 51\cos(1) + 49\sin(1) = 162.573447$

$y_3 = y_2 + h f(x_2, y_2) = -1.875718976 + \frac{1}{2}(162.573447) = 79.41100335 \rightarrow (x_3, y_3) = (1.5, 79.41100335)$

$\therefore x_4 = 2$

$f(x_3, y_3) = -50(79.41100335) + 51\cos(1.5) + 49\sin(1.5) = -3918.065316$

$y_4 = y_3 + h f(x_3, y_3) = 79.41100335 + \frac{1}{2}(-3918.065316) = -1879.621654 \rightarrow (x_4, y_4) = (2, -1879.621654)$

True solution: $Y(x) = \sin(x) + \cos(x)$ $Y(1) = 1.381773291$ $Y(2) = 0.491355903$

The notation to be utilized is x_n for the x value at the current step of the Euler method, y_n for the approximation at said x value, and $y(x_n)$ for the true value of the function at x_n . The latter values will be used to calculate errors.

Using the true solution at the bottom of the image for $x = 1$ and $x = 2$, the errors $y(x_n) - y_n$ can be computed to 3.257492267 and 1880.114806, respectively. These values can be found in the second and fourth row, respectively, of table 6.17 in Atkinson for $\lambda = -50$, $h = 0.5$. It should be mentioned that the table lists the x values themselves, not necessarily every step of the method.

“=====”

b)

The same problem was repeated with the *implicit* Euler method, the first four steps are below.

Implicit Euler Method $y_{n+1} = y_n + h f(x_{n+1}, y_{n+1})$ $f(x, y) = x y + (1-x) \cos y - (1-x) \sin x$, $y(0) = 1$
 $\lambda = -50$, $h = 0.5$ $x_0 = 0 \rightarrow f(x_0, y_0) = (0, 1)$
 $\rightarrow f(x, y) = -50 y + 51 \cos x + 49 \sin x$
 $\rightarrow y_n = y_{n+1} - h f(x_{n+1}, y_{n+1})$ $x_0 = 0, x_1 = 0.5, x_2 = 1, x_3 = 1.5, x_4 = 2$

i. $x_1 = 0 + 0.5 = 0.5$
 $y_0 = y_1 - 0.5 f(x_1, y_1) \rightarrow 1 = y_1 - 0.5 f(x_1, y_1) = y_1 - 0.5 (-50 y_1 + 51 \cos(0.5) + 49 \sin(0.5))$
 $\rightarrow 0 = 1 - 26 y_1 + \frac{51 \cos(0.5) + 49 \sin(0.5)}{2}$
 $\rightarrow 26 y_1 = 35.12423102$
 $y_1 = 1.35093398$ $(x_1, y_1) = (0.5, 1.35093398)$

ii. $x_2 = 0.5 + 0.5 = 1$
 $y_1 = y_2 - 0.5 f(x_2, y_2) \rightarrow 1.35093398 = y_2 - 0.5 (-50 y_2 + 51 \cos(1) + 49 \sin(1))$
 $-26 y_2 = \frac{51 \cos(1) + 49 \sin(1)}{2} - 1.35093398$
 $\rightarrow y_2 = 1.37479545$ $(x_2, y_2) = (1, 1.37479545)$

iii. $x_3 = 1 + 0.5 = 1.5$
 $y_2 = y_3 - 0.5 f(x_3, y_3) \rightarrow 1.37479545 = y_3 - 0.5 (-50 y_3 + 51 \cos(1.5) + 49 \sin(1.5))$
 $-26 y_3 = \frac{51 \cos(1.5) + 49 \sin(1.5)}{2} - 1.37479545$
 $\rightarrow y_3 = 1.06220082$ $(x_3, y_3) = (1.5, 1.06220082)$

iv. $x_4 = 1.5 + 0.5 = 2$
 $y_3 = y_4 - 0.5 f(x_4, y_4) \rightarrow 1.06220082 = y_4 - 0.5 (-50 y_4 + 51 \cos(2) + 49 \sin(2))$
 $-26 y_4 = \frac{51 \cos(2) + 49 \sin(2)}{2} - 1.06220082$
 $\rightarrow y_4 = 0.493150593$ $(x_4, y_4) = (2, 0.493150593)$

It is immediately clear that the approximations do not blow up as they did for the explicit Euler method, and they instead remain in a more reasonable range. While this does suggest that the implicit method performs better (at least in the context of this problem), the results must be analyzed further to make a justified conclusion.

c)

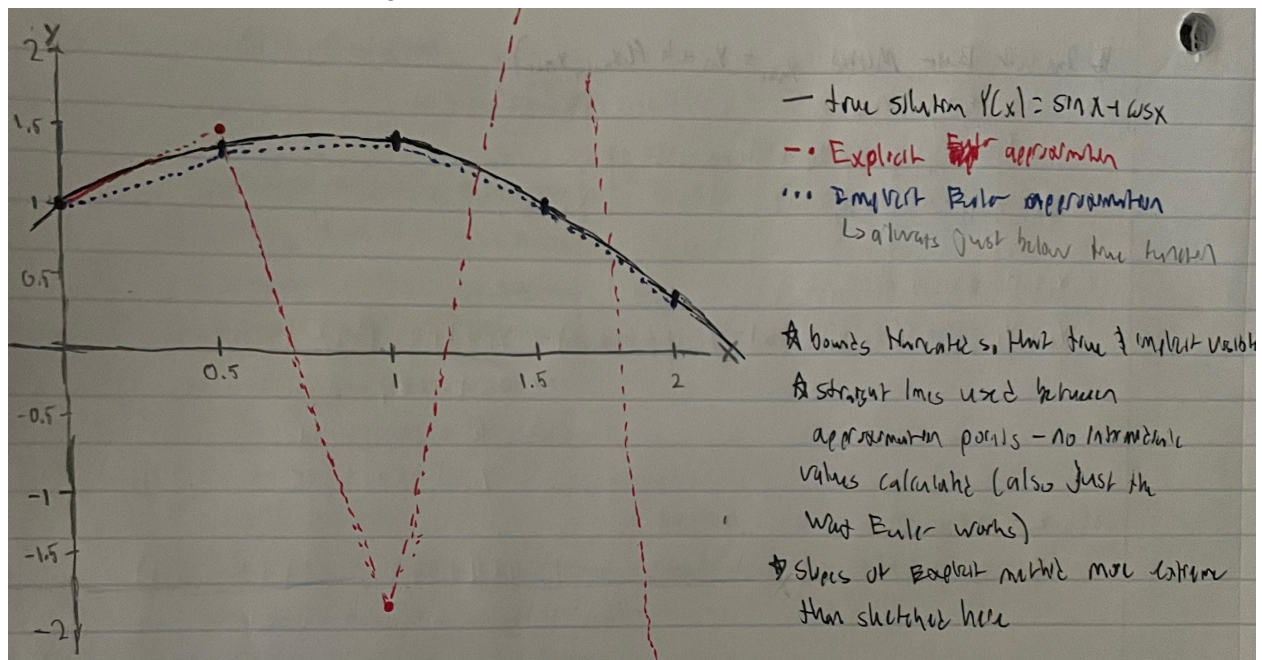
To compare the methods, the errors of each will be tabulated and included below.

Explicit Euler				
n	x_n	y_n	$y(x_n)$	$y(x_n) - y_n$
0	0	1	1	0
1	0.5	1.5	1.357008100	0.142918995
2	1	-1.87571896	1.381773291	3.257492267
3	1.5	79.41100335	1.068232188	78.34277116
4	2	-1879.621655	0.4931505903	1880.114806

Implicit Euler				
n	x_n	y_n	$y(x_n)$	$y(x_n) - y_n$
0	0	1	1	0
1	0.5	1.35093388	1.357008100	0.0060742204
2	1	1.37479545	1.381773291	0.0069778406
3	1.5	1.06220082	1.068232188	0.0060313682
4	2	0.48954782	0.4931505903	0.0036027702

It is clear that the implicit Euler method performs significantly better than the explicit method, as its errors do not blow up and stay close to 0 at each step. This is almost undoubtedly due to the fact that the implicit method does not impose a restriction on the value of h to maintain approximation stability.

To visualize the behavior of both methods with the chosen λ and step-size h , a plot of both numerical solutions along with the true solutions will be created.



While the implicit method clearly performs better, it is often significantly more difficult to implement. This is because the differential function may be non-linear in the unknown variable, at which point computer programs could be utilized to obtain solutions for functional values. In this example though, the functional values were easily solved-for by hand.

=====

“=====”

Problem 2

a)

The same problem will now be explored using Octave. The following code was utilized to obtain numerical solutions for the explicit Euler method, along with the true solutions. Below is the driver code. The first steps are to define the variables, $\lambda = -50$, $h = 0.5$, and timesteps x up to 2 along with the ODE as a function of x and y . Next, the Euler function, defined in a second file, is called which calculates the numerical solutions and returns them in a vector. The next step is to display the results using plotting functions to include axis labels and titles.

Importantly, the true solution $Y(x) = \sin x + \cos x$ is defined with a corresponding x vector so that it can be plotted against the approximation at each time step. To calculate and plot the errors, the true solution is evaluated at each of the time steps, and is then subtracted by the numerical solution (namely, the functional value y) at each corresponding time step.

Alex Caramanico May 2, 2023

```
1| lambda = -50;
2| f = @(x,y)(lambda*y + (1-lambda)*cos(x)-(1+lambda)*sin(x));
3| x = 0; y = 1; h = 0.5; span = [0 2]
4| [x,y] = euler(f,span,y,h)
5| # Plotting True Solution Against Euler's Method
6| xvec = [x:1/64:2]; # for plotting true solution
7| true = @(x)(sin(x)+cos(x)); # true solution
8| truevals = true(xvec); # true functional values
9| subplot(1,2,1)
10| H = plot(x,y,'-',xvec,truevals,'k');
11| set(H,'LineWidth',1);
12| axis([0 2 -2 2]);
13| grid on; xlabel('x'); ylabel('y'); legend("yn","Y(x)",'Location','southwest')
14| title(['lambda = ', num2str(lambda), ', h = ', num2str(h)]);
14| # Plotting Errors
15| steps = [x:h:2];
16| approx = [x,y](6:10);
17| errs = true(steps) - approx;
18| subplot(1,2,2)
19| plot(steps,abs(errs),'-r')
20| grid on; xlabel('x'); ylabel('y');
21| title(['Y(x)-yn for lambda = ', num2str(lambda), ', h = ', num2str(h)]);
22| [x,y] # results
```

The next code is written in the aforementioned separate file, so that the function can be called with other applicable problems in the future. The euler function has the following parameters:

- fhandle - the ordinary differential equation used to obtain numerical solutions
- tspan - the span determining the interval over which the true function is approximated
- y0 - initial value of y , previously shown to be exactly equal to the true initial value of $Y(x)$

- h - the distance between time steps

The first lines of the code establish the initial and ending time steps of the approximation, using the round function so that the number of time-steps is an integer. h is re-calculated so that the final time step is reached after exactly N steps. This is to say that lines 4 and 5 in the code prevent the time-steps from over or undershooting the endpoint b . Next, two empty vectors are defined with $N+1$ entries. This uses $N+1$ because both the initial point *and* endpoint are to be approximated with the method. The vectors are then updated with the initial values, after which the explicit Euler method is implemented via a for loop. The algorithm loops through each integer up to N , each of which corresponds to the current time step. The x and y vectors are updated by setting the next index according to the Explicit Euler procedure. Each successive entry in the vector for time steps x is separated by h , and each entry in the numerical solution vector y is defined by the explicit Euler formula $y_{n+1} = y_n + hf(x_n, y_n)$. The for loop ends when the final time step has been reached, at which point the time steps and numerical solutions are returned in the form of an appended vector.

```

1| function [x,y] = euler(fhandle,tspan,y0,h);
2| x0 = tspan(1);
3| b = tspan(end);
4| N = round((b-x0)/h); # round function so N is an integer
5| h = (b-x0)/N; # final time reached after N steps
6| x = zeros(1,N+1); # N+1, N time steps plus value at 0
7| y = zeros(1,N+1);
8| x(1) = x0; # initial time
9| y(1) = y0;
10| for n = 1 : N # loop to implement Euler method
11| x(n+1) = x0 + n*h; # update x value
12| y(n+1) = y(n) + h*fhandle(x(n),y(n)); # update y value, Explicit Euler
13| endfor
14| end;

```

The output on the following page shows the numerical results of the explicit Euler method at each corresponding time step, followed by two plots. The plot on the left shows the true solution in a solid black line against the numerical solution in a dash-dotted blue line. The plot on the right shows the absolute value of the errors at each time step.

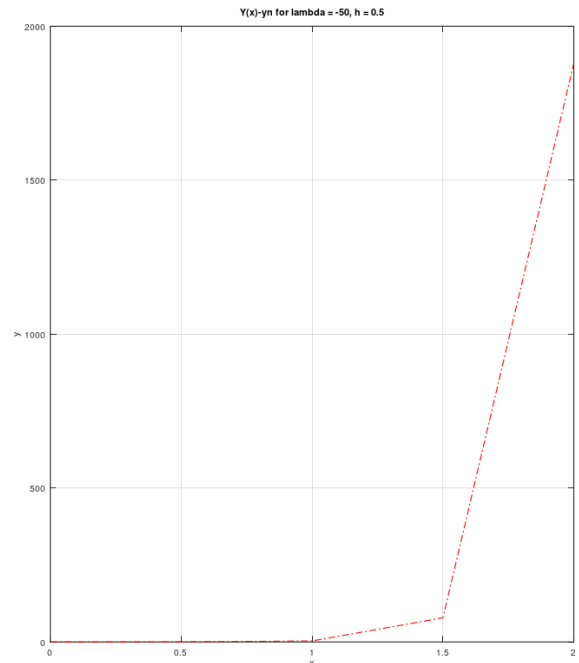
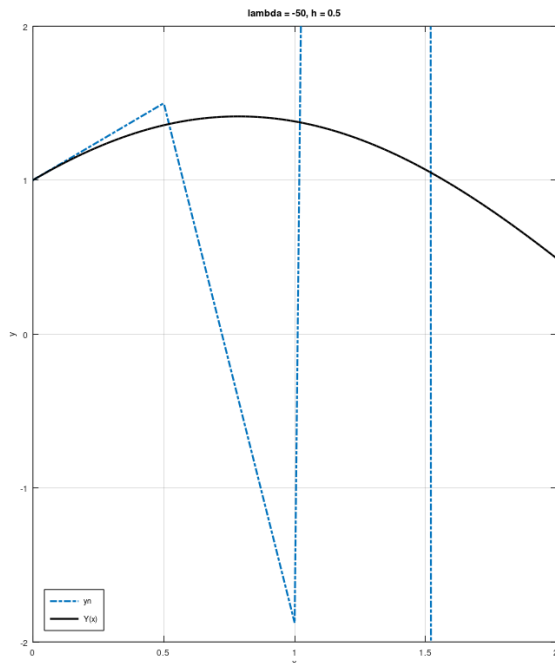
```

span =
    0    2

x =
    0    0.5000    1.0000    1.5000    2.0000

y =
    1.0000e+00    1.5000e+00   -1.8757e+00    7.9411e+01   -1.8796e+03

```



It is fairly clear that the explicit euler method does not perform well for this problem, because the approximation quickly blows up and misses the true value of the function by a significant amount. At only the third time step, the error is greater than 3 in absolute value. Future approximations are more inaccurate than previous ones, as the error plot shows exponential growth. That is, the error between the numerical solution and the true solution increases by more and more at each time step, and at an incredibly fast rate. This result was to be expected though, as larger values of λ in absolute value are known to be more stiff. In other words, h must be excessively small compared to the accuracy requirement. The error should be similar to h , and this problem contradicts that. Mathematically, stability of the approximations requires $h = \frac{1}{|\lambda|} = 0.02$. However, this exercise used $h = 0.5$. In any case, this h value would be much smaller than the error of approximation. It should also be noted that the function does *not* depend on λ , but if $h * \lambda$ is outside $[-2, 0]$, instability is to be expected.

“-----”

b)

Another means of understanding the problem at hand is to compare the numerical cost of methods. That is, the number of function evaluations for each method can be recorded, with more function evaluations suggesting a more costly method. All of the methods will be applied with time-steps in the span from 0 to 5, a larger endpoint than the previous exercises.

To execute this, the following Octave code was run, changing the value of “lambda” in the first line to -10 and -1 for each method to populate a table of results.

```
1| lambda = -50;
2| fode = @(x,y)(lambda*y + (1-lambda)*cos(x)-(1+lambda)*sin(x));
3| tspan = [0 5]; y0 = 1; options = odeset('Stats','on');
```

```

4| [x,y] = ode45(fode,tspan,y0,options);
5| plot(x,y);
6| title('Numerical Solutions'); xlabel('x'); ylabel('y')

```

Below is a table of the function evaluations made for three separate step-length values of Euler's method, three non-stiff methods, and four stiff methods. These non-stiff and stiff methods were implemented with pre-defined Matlab and Octave functions. It should be noted that methods ode113, ode23t and ode23tb are not defined in Octave, hence the number of their function evaluations could not be included.

Method	Function/h	$\lambda = -1$		$\lambda = -10$		$\lambda = -50$	
		Matlab	Octave	Matlab	Octave	Matlab	Octave
Euler	$h = 0.5$	10		10		10	
	$h = 0.1$	50		50		50	
	$h = 0.01$	500		500		500	
Non-Stiff	ode45	67	85	163	169	535	517
	ode23	64	85	124	133	448	397
	ode113	38	-	75	-	351	-
Stiff	ode15s	53	71	77	57	87	62
	ode23s	122	200	216	320	326	460
	ode23t	60	-	61	-	65	-
	ode23tb	68	-	59	-	35	-

Naturally, the number of function evaluations for Euler's method depends only on h . This is because h determines the number of time-steps, which is exactly equal to the number of function evaluations. The values in the table were calculated using the same formula as line 4 in the code for the euler function. That is, $N = (b - x_o)/h$, the number of function evaluations.

Naturally, smaller step sizes lead to a larger number of evaluations. While more evaluations would provide more accurate results when compared to the true solution, a very small step-size is extremely expensive computationally.

Of the non-stiff methods, ode113 is the least costly in Matlab as it requires the fewest function evaluations for each value of λ . For the stiff methods, ode15s, ode23t, and ode23tb have a similar number of evaluations regardless of the value of λ , while ode23s has significantly more for each λ . Along these lines is the observation that the number of evaluations for each Matlab method increases as $|\lambda|$ increases, aside from ode23tb, which has fewer approximations for larger values of $|\lambda|$. This is true for both the stiff methods *and* non-stiff methods. This can suggest that ode23tb is the most efficient method for the problem at hand, as it handles larger

values of λ better than any other method. In other words, the method upholds the linear stability analysis restrictions and is not particularly costly.

On the other hand, Octave performs similarly with its non-stiff methods, in that it evaluates the function a similar number of times to the non-stiff Matlab counterparts. Interestingly, Octave requires more functional evaluations at $\lambda = -1$, slightly more at $\lambda = -10$, but much less at $\lambda = -50$. The largest discrepancy between the Matlab and Octave methods can be seen with ode23s, where Octave requires significantly more evaluations at each value of λ . This implies that the documentation in Matlab is more efficient than Octave, at least for the ode23s method. The best performing Octave method, for that matter, is ode15s because it is the least costly regardless of λ .

“=====”